

APP DEVELOPMENT
Code Snippets & Technical Explanation

SUBMITTED TO:

SIR UZAIR HASSAN

SUBMITTED BY:

AABRU MUMTAZ

242201076

Department of Computer Science,
KICSIT Kahuta.

Contents

1 Project Architecture and File Structure 2

2 AndroidManifest.xml Configuration 2

3 SplashActivity Code and Explanation 3

4 SharedPreferences Emergency Contact Persistence 4

5 Voice SOS and Emergency Alert Implementation 4

6 AI Background Foreground Service Code 5

7 Conclusion 6

1 Project Architecture and File Structure

The **Silent SOS** Android project is built using Java and Android Studio XML layout files. The repository layout follows standard Android project architecture:

```
SilentSOS/
 app/
    src/
       main/
          java/com/example/silentsos/
             SplashActivity.java
             LoginActivity.java
             MainActivity.java
             ScreamDetectionService.java
          res/
             layout/
                activity_splash.xml
                activity_login.xml
                activity_main.xml
             values/
                 colors.xml
                 strings.xml
          AndroidManifest.xml
    build.gradle
```

2 AndroidManifest.xml Configuration

The `AndroidManifest.xml` file defines critical runtime permissions, components, foreground services, and the initial application launcher activity.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.silentsos">

    <!-- Permissions required for emergency functionality -->
    <uses-permission android:name="android.permission.SEND_SMS" />
    <uses-permission android:name="android.permission.
   ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.
   FOREGROUND_SERVICE" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="Silent SOS"
        android:theme="@style/Theme.AppCompat.Light.NoActionBar">

        <!-- Launcher Splash Activity -->
        <activity android:name=".SplashActivity" android:exported="true
   ">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

```
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".LoginActivity" />
        <activity android:name=".MainActivity" />

        <!-- AI Audio Background Service -->
        <service android:name=".ScreamDetectionService"
    android:exported="false" />

    </application>
</manifest>
```

Listing 1: AndroidManifest.xml Snippet

Technical Explanation:

- `SEND_SMS` & `ACCESS_FINE_LOCATION`: Enables the app to dispatch distress SMS messages containing live location data during emergencies.
- `RECORD_AUDIO` & `FOREGROUND_SERVICE`: Required by the AI Scream Detection module to continuously monitor environmental sound in the background.

3 SplashActivity Code and Explanation

`SplashActivity.java` manages the initial startup experience, displaying the branded logo for 2.5 seconds before navigating to the login screen.

```
package com.example.silentsos;

import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import androidx.appcompat.app.AppCompatActivity;

public class SplashActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_splash);

        // 2.5-second delayed handler transition
        new Handler().postDelayed(new Runnable() {
            @Override
            public void run() {
                Intent intent = new Intent(SplashActivity.this, LoginActivity.class);
                startActivity(intent);
                finish();
            }
        }, 2500); // 2500 milliseconds delay
    }
}
```

Listing 2: SplashActivity.java Snippet

Technical Explanation: A `Handler` object posts a runnable to the message queue with a 2500ms delay. When executed, it launches `LoginActivity` and calls `finish()` so the user cannot return to the splash screen via the back button.

4 SharedPreferences Emergency Contact Persistence

In `MainActivity.java`, the emergency guardian contact is managed using `SharedPreferences` to ensure permanent local storage across app restarts.

```
private SharedPreferences sharedPreferences;
private static final String PREF_NAME = "SilentSOSPrefs";
private static final String KEY_GUARDIAN_NUMBER = "guardian_number";

// Checking and saving emergency contact
private void checkAndPromptEmergencyContact() {
    sharedPreferences = getSharedPreferences(PREF_NAME, MODE_PRIVATE);
    String savedNumber = sharedPreferences.getString(
   KEY_GUARDIAN_NUMBER, null);

    if (savedNumber == null || savedNumber.isEmpty()) {
        showSetContactDialog();
    } else {
        updateContactDisplay(savedNumber);
    }
}

private void saveGuardianNumber(String number) {
    SharedPreferences.Editor editor = sharedPreferences.edit();
    editor.putString(KEY_GUARDIAN_NUMBER, number);
    editor.apply(); // Asynchronous save
    updateContactDisplay(number);
}
```

Listing 3: SharedPreferences Setup in MainActivity.java

Technical Explanation: `SharedPreferences` accesses local XML key-value storage. If `KEY_GUARDIAN_NUMBER` is empty, a dialog pops up. Upon saving, `editor.apply()` writes the data asynchronously to avoid blocking the UI thread.

5 Voice SOS and Emergency Alert Implementation

`MainActivity.java` contains speech recognition integration and the one-tap emergency SMS trigger.

```
// Launching Google Speech Recognizer
private void startVoiceSOS() {
    Intent intent = new Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH
   );
    intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
   RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
    intent.putExtra(RecognizerIntent.EXTRA_PROMPT, "Speak Emergency
   Message...");
    try {
        startActivityForResult(intent, VOICE_REQUEST_CODE);
    } catch (Exception e) {
```

```
        Toast.makeText(this, "Speech Recognition Not Supported", Toast.
    LENGTH_SHORT).show();
    }
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent
    data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == VOICE_REQUEST_CODE && resultCode == RESULT_OK &&
    data != null) {
        ArrayList<String> result = data.getStringArrayListExtra(
    RecognizerIntent.EXTRA_RESULTS);
        if (result != null && !result.isEmpty()) {
            String spokenText = result.get(0);
            sendEmergencySMS("VOICE SOS: " + spokenText);
        }
    }
}
```

Listing 4: Voice SOS Speech-to-Text Integration

Technical Explanation: Google's `RecognizerIntent` captures spoken audio and returns transcribed text in `onActivityResult`. This string is immediately forwarded to the `sendEmergencySMS()` method.

6 AI Background Foreground Service Code

`ScreamDetectionService.java` runs as an Android Foreground Service to perform background audio monitoring.

```
package com.example.silentsos;

import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.Service;
import android.content.Intent;
import android.os.Build;
import android.os.IBinder;
import androidx.core.app.NotificationCompat;

public class ScreamDetectionService extends Service {
    private static final String CHANNEL_ID = "ScreamDetectionChannel";

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        createNotificationChannel();
        Notification notification = new NotificationCompat.Builder(this
    , CHANNEL_ID)
                .setContentTitle("Silent SOS Active")
                .setContentText("AI Scream Detection is running in
    background...")
                .setSmallIcon(R.drawable.ic_launcher_foreground)
                .build();

        startForeground(1, notification);
```

```
25          startAudioMonitoring();
26          return START_STICKY;
27      }
28
29      private void startAudioMonitoring() {
30          // AudioRecord buffer initialization and TFLite audio
    processing logic
31      }
32
33      @Override
34      public IBinder onBind(Intent intent) {
35          return null; // Unbound service
36      }
37  }
```

Listing 5: ScreamDetectionService.java Foreground Service

Technical Explanation: The service calls `startForeground()` with a persistent status bar notification. Returning `START_STICKY` ensures that if the system kills the service due to low memory, it is automatically restarted as soon as resources become available.

7 Conclusion

This technical document concludes the code snippet review for the **Silent SOS** project. All code components—from user authentication and local storage to speech recognition and background AI monitoring services—have been modularized, documented, and tested for execution.